

Coding Assignment 5: The Waiting Room That Wraps Around

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

September 17, 2026

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).

5.1 Why this problem

Week 5 gave the calculator a waiting room: jobs leave in the order they arrived, the buffer never walks off the end of its array, and urgent jobs jump the line. The lecture stated both contracts but left the *code* for you to write, with one deliberate simplification you must now unlearn: on the slides, `dequeue` on an empty queue returns `-1` to keep the slide readable. That sentinel cannot tell “the queue held `-1`” from “the queue was empty” — a real queue must say which happened. This assignment is those two real structures: Lab P5 (a circular queue that reuses freed slots by wraparound) and Lab P6 (an unordered priority queue that scans for the minimum), both running on your array implementations with explicit status codes throughout.

5.2 Your task

Implement two ADTs in C as **fixed arrays with explicit status codes on every fallible operation**. The contracts are given to you in `queue.h` — *do not modify it*. Copy `starter_queue.c` to `queue.c` and fill in all fifteen functions:

```
void    cqueue_init(CQueue *q);
int      cqueue_is_empty(const CQueue *q);    /* 1 iff count == 0 */
int      cqueue_is_full(const CQueue *q);     /* 1 iff count == QUEUE_MAX */
int      cqueue_size(const CQueue *q);
CQStatus cqueue_enqueue(CQueue *q, int x);    /* CQ_OK / CQ_FULL */
CQStatus cqueue_dequeue(CQueue *q, int *out); /* CQ_OK / CQ_EMPTY */
CQStatus cqueue_front(const CQueue *q, int *out);
int      cqueue_display(const CQueue *q, int *out, int out_cap);

void    pqueue_init(PQueue *q);
int      pqueue_is_empty(const PQueue *q);
int      pqueue_is_full(const PQueue *q);
int      pqueue_size(const PQueue *q);
PQStatus pqueue_enqueue(PQueue *q, int value, int priority);
PQStatus pqueue_peek_min(const PQueue *q, int *value_out, int *priority_out);
PQStatus pqueue_delete_min(PQueue *q, int *value_out, int *priority_out);
```

P5 is the count version from lecture. `cqueue_init` sets `front` to 0, `rear` to `-1`, `count` to 0, so the first enqueue lands at index 0. Every enqueue advances `rear = (rear + 1) mod 8` and increments `count`; every dequeue reads `data[front]` *before* advancing `front = (front + 1) mod 8` and decrementing `count`. Full iff `count == 8` — never test `front == rear`, which also fires on an empty ring. `display` copies the live items front-to-rear into the caller’s buffer and returns how many it wrote (`-1` when the buffer is too small); it never disturbs the queue.

P6 is the unordered design from lecture. `enqueue` appends at the end in $O(1)$. `peek_min` and `delete_min` scan every live item and return the *smallest priority number* first (urgency 0 beats urgency 9); ties leave in arrival order, earliest-inserted first. `delete_min` closes the gap so the survivors stay packed. Either out pointer may be `NULL` (discard that half); on *any* non-OK status, the queue and every `*out` are left unchanged.

5.3 Constraints

- Representation must be the `CQueue/PQueue` arrays in `queue.h` — no linked lists, no heap allocation, no global/static mutable state.
- Error paths use the explicit status codes — never the lecture’s `-1` sentinel. A submission whose `dequeue` returns `-1-as-error` fails the hidden suite’s value-trap tests (enqueueing the *value* `-1` and reading it back must succeed with `CQ_OK`).
- Correctness is graded against the contract; your code must compile with `gcc -Wall -Wextra` without errors or warnings.
- Complexity: circular `enqueue`, `dequeue`, `front` are $O(1)$ worst case each, holding at most `QUEUE_MAX` integers; priority `enqueue` is $O(1)$ worst case while `peek_min` and `delete_min` each scan all live items at $O(n)$ worst case in the current size.

5.4 Worked examples (these are the visible tests)

Example 1 — the circular queue itself. After `init`: empty, size 0. `enqueue(5)`, `enqueue(15)`, `enqueue(25)` all return `CQ_OK`; `front` gives 5 and size stays 3; `dequeue` gives 5, then 15; size is 1 and `front` is 25.

Example 2 — wraparound, display, honest full/empty (P5). Enqueue 11, 12, 13, 14, 15, 16; dequeue 11, 12; enqueue 17, 18, 19 — the last one walks `rear` from index 7 back to index 0. Size is 7, not full. `display` writes 7 items in front-to-rear order: 13, 14, 15, 16, 17, 18, 19. `enqueue(20)` fills the ring (full at 8 of 8); `enqueue(21)` returns `CQ_FULL`. Draining gives 13, 14, 15, 16, 17, 18, 19, 20 in order; the queue is empty and the next `dequeue` returns `CQ_EMPTY`.

Example 3 — the priority runner (P6). Enqueue (701, 4), (702, 1), (703, 3). `peek_min` reports (702, 1) and size stays 3. `delete_min` returns 702, then 703. Enqueue (704, 1); `delete_min` returns 704, then 701; the queue is empty and the next `delete_min` returns `PQ_EMPTY`. Ties keep arrival order: enqueueing (801, 2) then (802, 2) deletes 801 first.

5.5 What to submit

1. `queue.c` — your completed implementation (edit `starter_queue.c`).
2. `queue.h` — unchanged, but include it so the submission compiles standalone.

5.6 Getting started

1. Copy `starter_queue.c` to `queue.c` and fill in the fifteen function bodies.
2. Sanity-check locally:

```
gcc -Wall -Wextra -o test_visible test_visible.c queue.c
./test_visible
```

All three worked examples should pass.

3. Submit `queue.c` and `queue.h`. The autograder compiles your `queue.c` against a *hidden* test suite and reports pass/fail. The hidden suite covers: the `-1` value trap and `INT_MIN/INT_MAX` round-trips; filling the ring to exactly `QUEUE_MAX` and one past it; `front` purity and `NULL`-out calls; display with too-small and `NULL` buffers (with a canary check that nothing is written on refusal); wraparound states including the count-honesty configuration that defeats any `front == rear` full test; repeated fill/drain cycles; negative and tied priorities with earliest-wins ordering; peeking/deleting through `NULL` halves; filling the priority queue to `PQUEUE_MAX`; a twenty-item delete-all-sorted check; and seeded random campaigns for both structures checked operation-by-operation against independent oracles.